

DFS, BFS and their applications

Graph Algorithms

Diptapriyo Majumdar

March 13, 2023

Outline

① Recalling Directed DFS

② Longest Path in DAG

```

DFS( $G$ ) for every  $u \in V(G)$ 
  do
    | Mark  $u$  as unvisited;
    |  $parent(u) = NIL$ ;
  end
for every  $u \in V(G)$  do
  | if  $u$  is unvisited then
  | | DFS-Visit( $G, u$ );
  | end
end

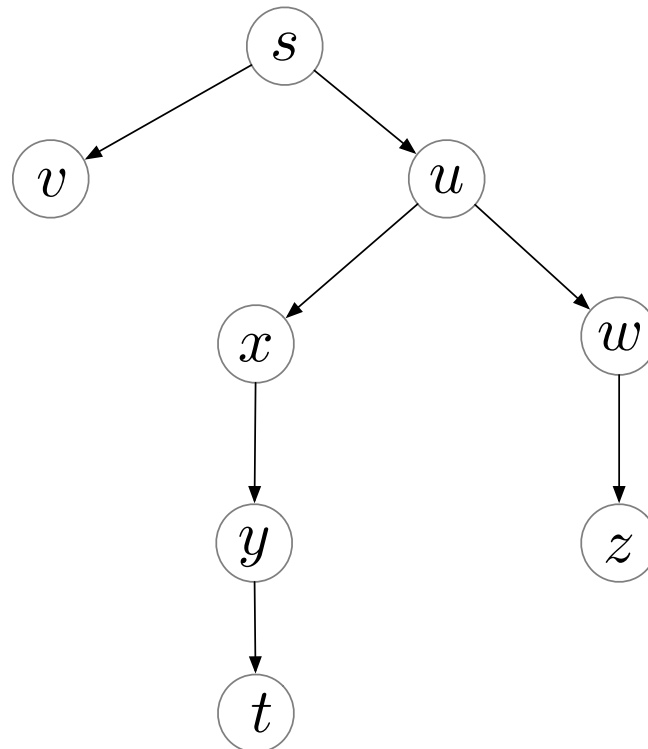
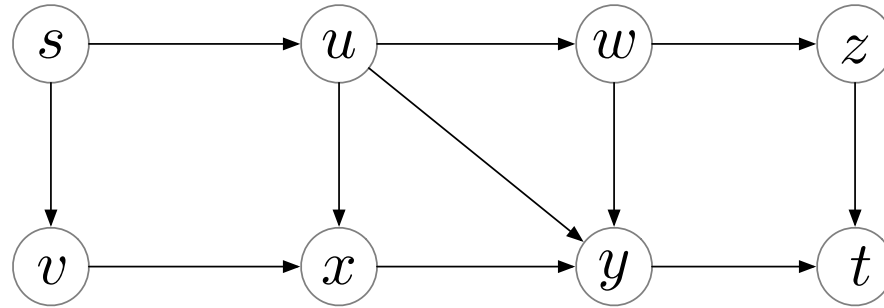
```

```

DFS-Visit( $G, s$ )
 $time \leftarrow time + 1$ ;  $start[s] = time$ ;
Mark  $s$  visited; Color  $u$  with grey;
for every  $u \in Adj(s)$  do
  | if  $u$  is unvisited then
  | |  $parent(u) = s$ ;
  | | DFS-Visit( $G, u$ );
  | end
end
Mark  $s$  as explored; Color  $s$  black;
 $time \leftarrow time + 1$ ;  $finish[s] = time$ ;

```

Example



Some facts related to DFS

Let the DFS procedure starts from s . Then,

- DFS yields a tree rooted at s , called **DFS-Tree**.
- **Tree Edge:** The edges of the graph that are there in the DFS-Tree.
- **Forward Edge:** An edge (u, v) such that v is a descendant of u in the DFS-Tree.
- **Back Edge:** An edge $(v, u) \in E(G)$ such that v is a descendent of u in the DFS-Tree.
- **Cross Edge:** An edge $(u, v) \in E(G)$ such that u and v are in different subtrees of s in the DFS-Tree.

Some facts related to DFS

Let the DFS procedure starts from s . Then,

- DFS yields a tree rooted at s , called **DFS-Tree**.
- **Tree Edge:** The edges of the graph that are there in the DFS-Tree.
- **Forward Edge:** An edge (u, v) such that v is a descendant of u in the DFS-Tree.
- **Back Edge:** An edge $(v, u) \in E(G)$ such that v is a descendent of u in the DFS-Tree.
- **Cross Edge:** An edge $(u, v) \in E(G)$ such that u and v are in different subtrees of s in the DFS-Tree.
- A visited neighbor v of u is either parent of u or (u, v) is a back edge.
- If a vertex is marked explored, then all the vertices in its subtrees are also marked explored.

Some facts related to DFS

Let the DFS procedure starts from s . Then,

- DFS yields a tree rooted at s , called **DFS-Tree**.
- (u, v) is a **tree edge** if $start[u] < start[v] < finish[v] < finish[u]$ and $u = parent(v)$.
- (u, v) is a **forward edge**: if $start[u] < start[v] < finish[v] < finish[u]$ but $u \neq parent(v)$.
- (u, v) is a **back edge** if $start[v] \leq start[u] < finish[u] \leq finish[v]$.
- (u, v) is a **cross edge** if $start[v] < finish[v] < start[u] < finish[u]$.

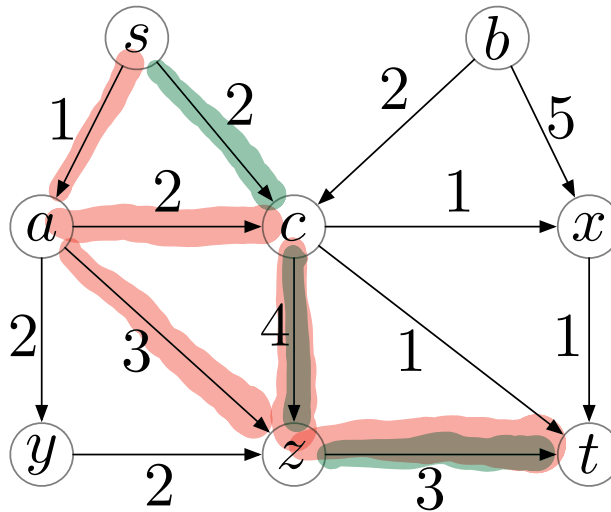
Outline

① Recalling Directed DFS

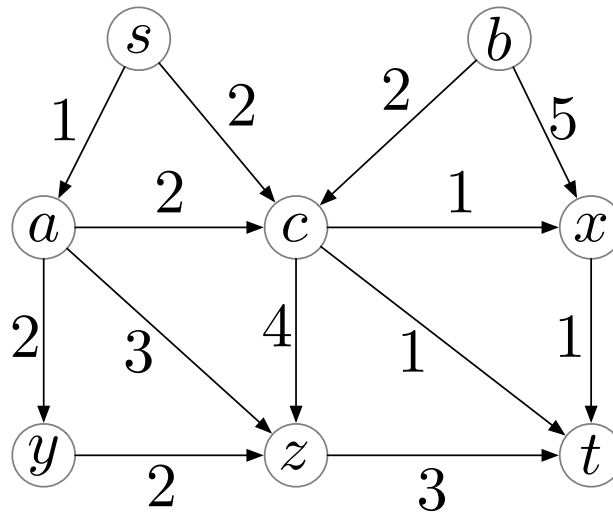
② Longest Path in DAG

Longest Path in DAG

- **Input:** An edge-weighted directed acyclic graph (DAG) $G = (V, E)$ and vertices s and t .
- **Output:** The total weight of a longest path from s to t in G .

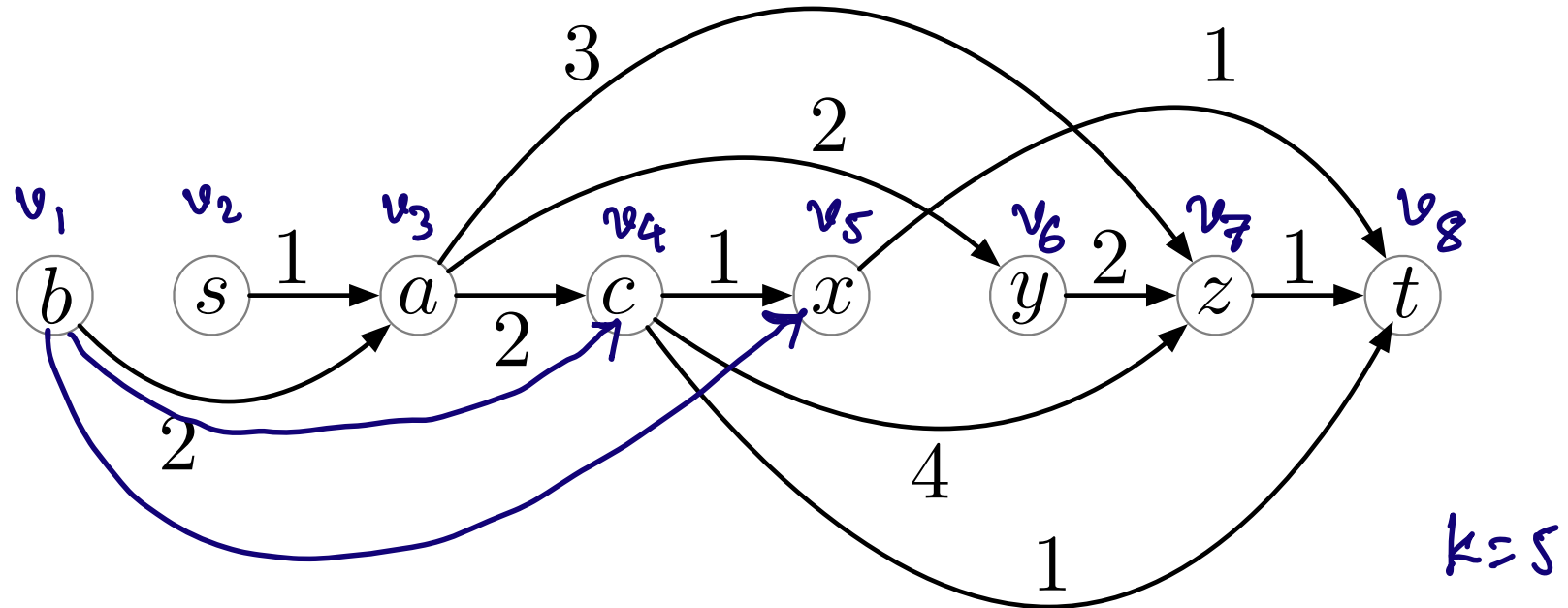


9
7



s, b, a, c, x, y, z, t
 $v_1, v_2, v_3, v_4, v_5, v_6, v_7$

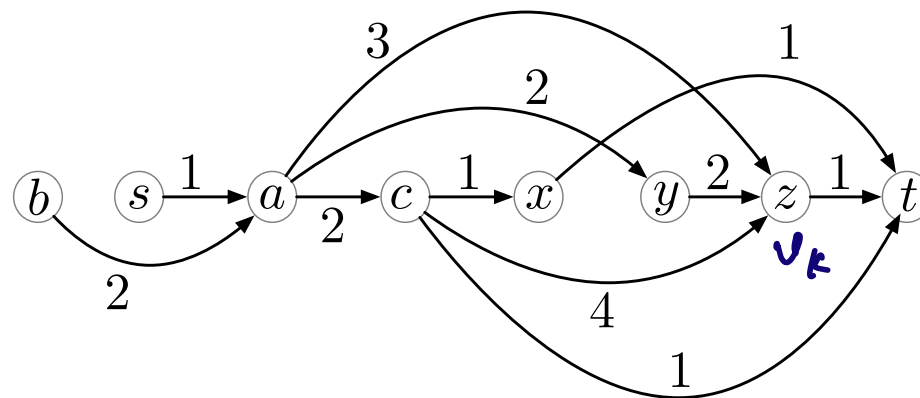
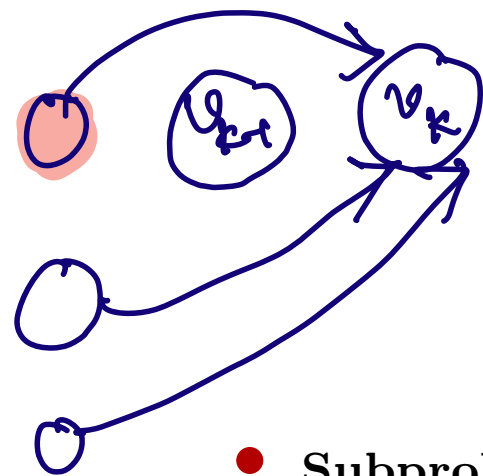
- ✓ A topological ordering of a DAG G is an ordering of the vertices $(v_1, \dots, v_n)$ such that any edge (v_i, v_j) of G must satisfy $i < j$.
- ✓ Any directed acyclic graph (DAG) admits a topological ordering.
- Given a directed acyclic graph G , a topological ordering of G can be computed in $O(|V| + |E|)$ -time.



- **Preprocessing Stage:** Compute a topological order $\sigma = (v_1, \dots, v_n)$.

- **Subproblem Definition:** $\text{PATH}(v_k)$.

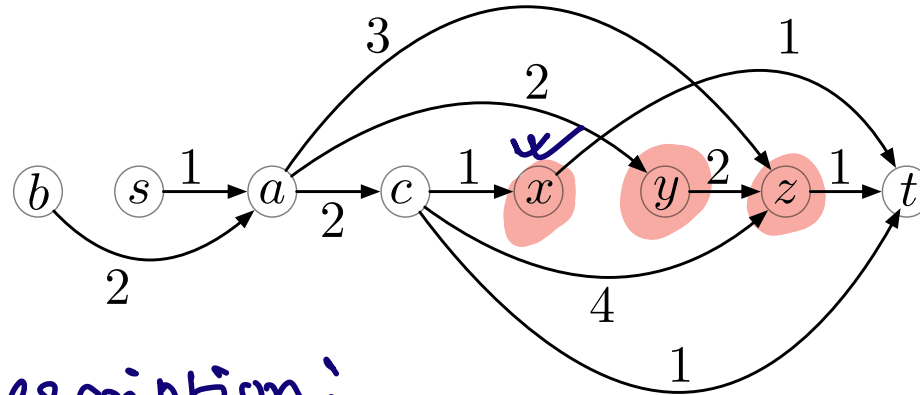
In the graph $G[\{v_1, v_2, \dots, v_k\}]$ the total weight of a longest path from s to v_k .



S_k = the vertices from which there is an incoming edge into v_k .

- Subproblem Definition: $\text{PATH}(k)$. the total weight of a longest path from s to v_k in $G[\{v_1, \dots, v_k\}]$.
- Recurrence Relation: $s = v_i$
 - $\{ \text{PATH}(k) = -\infty \text{ for all } k \leq i-1. \}$ Base Cases
 - $\text{PATH}(s) = \text{PATH}(i) = 0$
 - $\text{PATH}(k) = \max_{u: (u, v_k) \in E} [\text{PATH}(u) + \text{Weight}(u, v_k)]$
- Subproblem that solves the actual problem: $\text{PATH}(t)$

Algorithm Outline



Algorithm Description:

- $\sigma = (v_1, \dots, v_n) \leftarrow$ a topological ordering of G .
- For all $v \in V \setminus \{s\}$, initialize $PATH(v) \leftarrow -\infty$ and $PATH(s) = 0$.
- For all $1 \leq i \leq n$

$$PATH(v_i) = \max_{(u, v_i) \in E} \{PATH(u) + Weight(u, v_i)\}$$

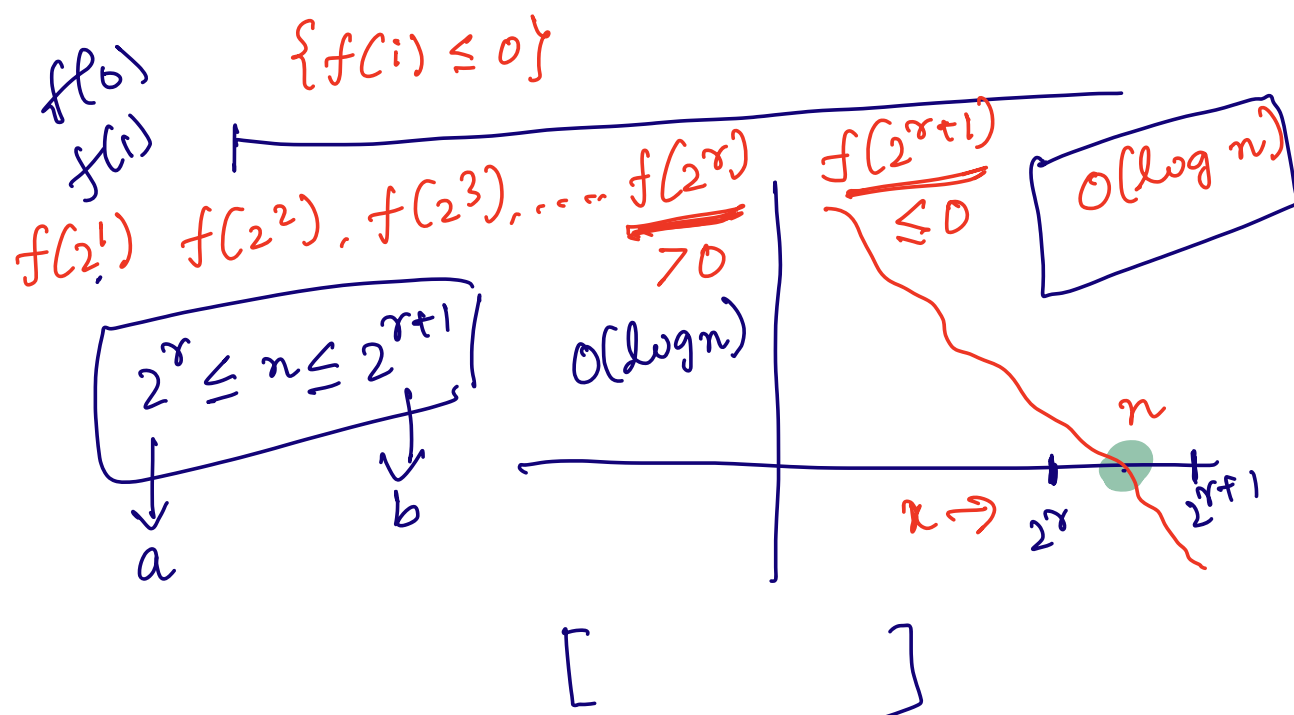
- Output $PATH(t)$

$O(|V| + |E|)$

$O(|V|)$

$deg(v_i)$

$O(|E|)$



$$\binom{n}{k} = {}^nC_k$$

$$\frac{n!}{k!(n-k)!}$$

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$